

PRACTICA N°4 - PLANIFICACIÓN DE PROCESOS.

1. Responda en forma sintética sobre los siguientes conceptos:

A. Programa y Proceso.

Un programa es estático, no posee un PC y existe desde que se edita hasta que se borra.

Un proceso es dinámico, posee PC y su ciclo de vida comprende desde que se lo ejecuta hasta que finaliza.

B. Defina Tiempo de retorno (TR) y Tiempo de espera (TE) para un Job.

TR (Tiempo de retorno) : Es el tiempo que transcurre desde que el proceso es cargado en memoria , se le otorga cpu para ejecutarse y finaliza por completo.

TE (Tiempo de espera) : Es el tiempo de retorno menos el tiempo de cpu (Ejecución), el tiempo de espera de un proceso a ser ejecutado.

C. Defina Tiempo Promedio de Retorno (TPR) y Tiempo promedio de espera (TPE) para un lote de JOBS. **COMPLETAR**

D. ¿Qué es el Quantum?

El quantum es el tiempo máximo que un proceso puede ejecutarse en la CPU antes de que el sistema operativo lo interrumpa para dar tiempo a otros procesos

El "quantum" determina la cantidad de tiempo que se asigna a cada proceso antes de que se produzca un cambio de contexto. Si un proceso no termina de ejecutarse dentro de su quantum, se suspende y se pone en cola para que otro proceso tenga la oportunidad de ejecutarse. Esto ayuda a asegurar que todos los procesos reciban un tiempo razonable de CPU y mejora la respuesta del sistema..

E. ¿Qué significa que un algoritmo de scheduling sea apropiativo o no apropiativo (Preemptive o Non-Preemptive)?

Se refiere a cómo se manejan los procesos en ejecución en un sistema operativo.

Algoritmos Apropiativos (Preemptive)

- En un algoritmo apropiativo, el sistema operativo puede interrumpir un proceso en ejecución para asignar la CPU a otro proceso. Esto significa que un proceso puede ser detenido antes de que termine su ejecución (llevándolo a la cola de listos- ready), permitiendo que otros procesos puedan ser atendidos.

Algoritmos No Apropiativos (Non-Preemptive)

- En un algoritmo no apropiativo, una vez que un proceso comienza a ejecutarse, no puede ser interrumpido hasta que termine su ciclo de ejecución o se bloquea por algún evento como de i/o . El sistema solo cambia a otro proceso cuando el actual ha completado su tarea.

F. ¿Qué tareas realizan?:

> SHORT TERM SCHEDULER : Es el encargado de asignar cpu a los procesos que estén listos para ejecutarse.

> MEDIUM TERM SCHEDULER: Realiza el swapping(interchange) entre el disco y la memoria cuando el SO lo determina. Para que se libere la memoria y nuevos procesos puedan ser cargados.

> LONG TERM SCHEDULER : Es el encargado de admitir nuevos procesos a memoria.

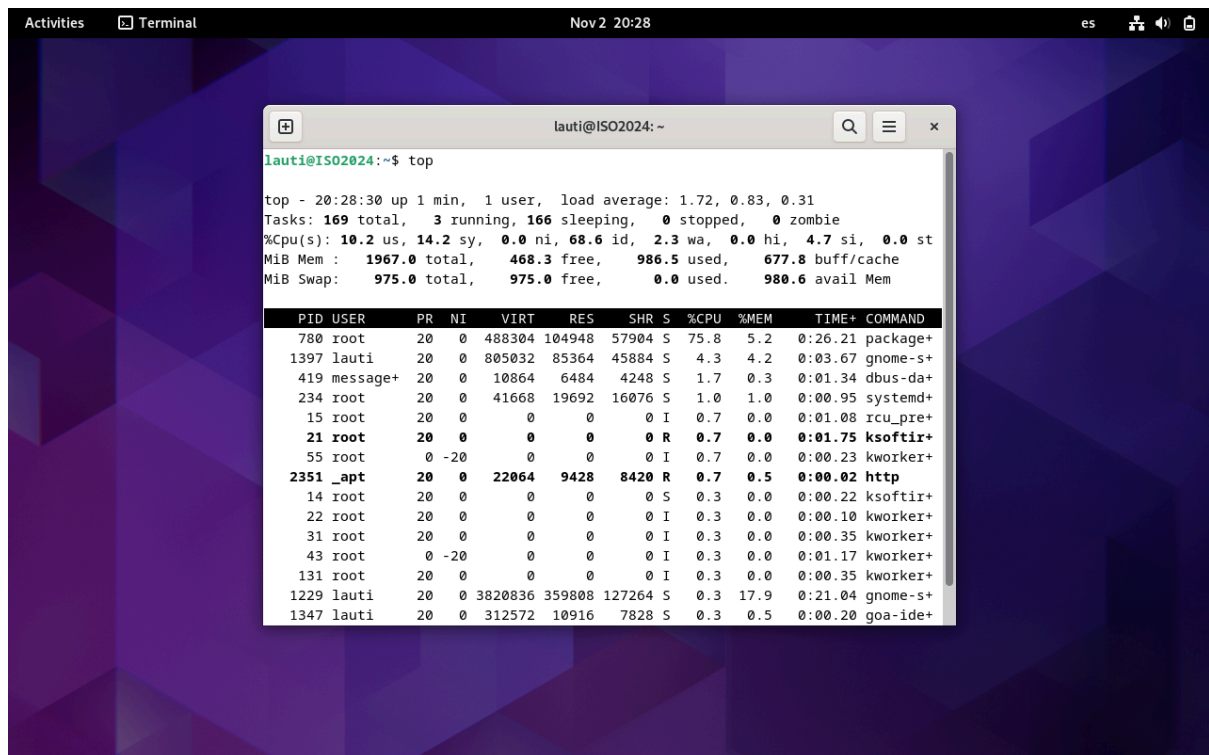
G. ¿Qué tareas realiza el Dispatcher? **TERMINAR**

El *dispatcher* es un componente clave del sistema operativo que se encarga de gestionar la asignación de la CPU a los procesos.

2. Procesos :

A. Investigue y detalle para que sirve cada uno de los siguientes comandos.

> top : Muestra en tiempo real el uso de recursos del sistema, incluyendo la CPU, la memoria, y los procesos en ejecución.



```
lauti@ISO2024:~$ top

top - 20:28:30 up 1 min, 1 user, load average: 1.72, 0.83, 0.31
Tasks: 169 total, 3 running, 166 sleeping, 0 stopped, 0 zombie
%Cpu(s): 10.2 us, 14.2 sy, 0.0 ni, 68.6 id, 2.3 wa, 0.0 hi, 4.7 si, 0.0 st
MiB Mem : 1967.0 total, 468.3 free, 986.5 used, 677.8 buff/cache
MiB Swap: 975.0 total, 975.0 free, 0.0 used, 980.6 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR  S  %CPU  %MEM    TIME+  COMMAND
  780 root        20   0 488304 104948 57904 S   75.8   5.2   0:26.21 package+
 1397 laut      20   0 805032 85364 45884 S    4.3   4.2   0:03.67 gnome-s+
  419 message+ 20   0 10864  6484  4248 S    1.7   0.3   0:01.34 dbus-da+
  234 root       20   0 41668 19692 16076 S    1.0   1.0   0:00.95 systemd+
   15 root       20   0      0      0      0 I    0.7   0.0   0:01.08 rcu_pre+
   21 root       20   0      0      0      0 R    0.7   0.0   0:01.75 ksoftir+
   55 root       20  -20      0      0      0 I    0.7   0.0   0:00.23 kworker+
 2351 _apt       20   0 22064  9428  8420 R    0.7   0.5   0:00.02 http
   14 root       20   0      0      0      0 S    0.3   0.0   0:00.22 ksoftir+
   22 root       20   0      0      0      0 I    0.3   0.0   0:00.10 kworker+
   31 root       20   0      0      0      0 I    0.3   0.0   0:00.35 kworker+
   43 root       20  -20      0      0      0 I    0.3   0.0   0:01.17 kworker+
  131 root       20   0      0      0      0 I    0.3   0.0   0:00.35 kworker+
 1229 laut      20   0 3820836 359808 127264 S    0.3  17.9   0:21.04 gnome-s+
 1347 laut      20   0 312572 10916  7828 S    0.3   0.5   0:00.20 goa-ide+
```

- > htop : Similar a **top**, pero con una interfaz más amigable y visual. Permite una interacción más fácil, como desplazarse y seleccionar procesos.
- > ps : Muestra una instantánea de los procesos en ejecución en el sistema. Comúnmente se utiliza con opciones como **ps aux** para mostrar todos los procesos o **ps -ef** para un formato diferente. Es útil para identificar procesos y su estado.
- > pstree : Muestra los procesos en forma de árbol, mostrando la jerarquía y la relación entre ellos.
- > kill : Envía señales a procesos, comúnmente para terminarlos. Usualmente se utiliza con el PID (identificador de proceso), como **kill <PID>**.
- > pgrep : Busca procesos en ejecución que coincidan con ciertos criterios, como nombre o atributos.
- > pkill : Similar a **kill**, pero permite terminar procesos basándose en criterios de búsqueda (como el nombre).
- > killall : Envía una señal a todos los procesos que coinciden con un nombre específico. Por ejemplo, **killall firefox** terminará todos los procesos de Firefox.
- > renice : Permite cerrar ventanas gráficas haciendo clic en ellas.

> atop : Monitorea y registra el uso de recursos del sistema a lo largo del tiempo, similar a **top** pero con más detalles.

B. Observe detenidamente el siguiente código. Intente entender lo que hace sin necesidad de ejecutarlo.

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main(void) {
    int c;
    pid_t pid;
    printf("Comienzo.:\n");
    for (c = 0; c < 3 ; c++ )
    {
        pid = fork();
    }
    printf("Proceso\n");
    return 0;
}
```

i. ¿Cuántas líneas con la palabra “Proceso” aparecen al final de la ejecución de este programa?.

Dos

ii. ¿El número de líneas es el número de procesos que han estado en ejecución?. Ejecute el programa y compruebe si su respuesta es correcta, Modifique el valor del bucle for y compruebe los nuevos resultados.

5. Se tiene el siguiente lote de procesos que arriban al sistema en el instante 0 (cero):

Job	Unidades de CPU
1	7
2	15
3	12
4	4
5	9

Punto A y B resuelto en Excel.

C. En base a los tiempos calculados compare los diferentes algoritmos.

El promedio más chico fue el del algoritmo SJF con :

	TR	TE	
	11	4	
	47	32	
	32	20	
	4	0	
	20	11	
	22.8	13.4	

